

CAPÍTULO 8

CAPÍTULO 8 – Persistencia de Datos

Objetivos de hoy:

- Entender que es persistir datos
- Crear archivos de texto en disco duro para recuperar datos
- Grabar líneas en un archivo
- Leer líneas de un archivo
- Utilizar la Clase file

¿Qué es Persistencia de Datos?

- La **persistencia de datos** es la capacidad de un equipo para mantener la información incluso después de apagar el ordenador o cerrar un programa que utiliza la información.

- Persistencia de los datos en esencia conserva el estado de la información en un momento en que usted elija, lo guarda, por ejemplo, cuando hace clic en un botón "Guardar" o copiar archivos en un dispositivo de almacenamiento no volátil.

- Una de las maneras más simples de mantener sus datos es a través de archivos de texto. Otra alternativa es usar Bases de Datos

Beneficios

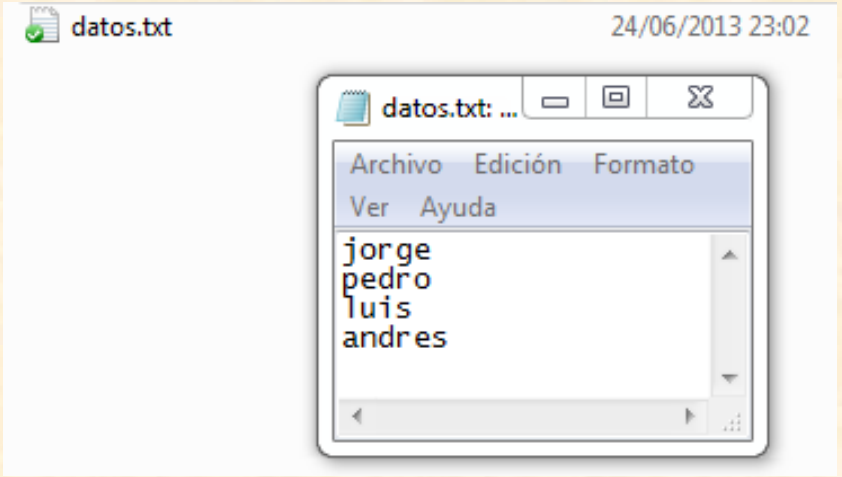
- La mayoría de los programas de computadora usan la memoria RAM para almacenar temporalmente la información que utiliza el programa, tales como datos de la función o referencias a variables. Al cerrar el programa, el equipo se borran todos los datos en la RAM. Al persistir los datos sobre archivos de texto, permite que los datos con los que se trabajan no se pierdan.

- Los datos persistentes pueden ser recuperados, modificados y almacenados nuevamente, conservando los cambios realizados de manera permanente.

2.- Archivos de Textos

- Un archivo de texto contiene una sucesión de caracteres que podemos considerar organizada en una secuencia de líneas.

- Los programas Python, por ejemplo, suelen residir en archivos de texto. Es posible generar, leer y modificar archivos de texto con editores de texto o con nuestros propios programas.



Archivos de Textos

- Desde el punto de vista de la programación un archivo de texto es un objeto en el que se puede leer y/o escribir información. Esto obliga a que siempre debamos seguir un protocolo establecido de tres pasos:

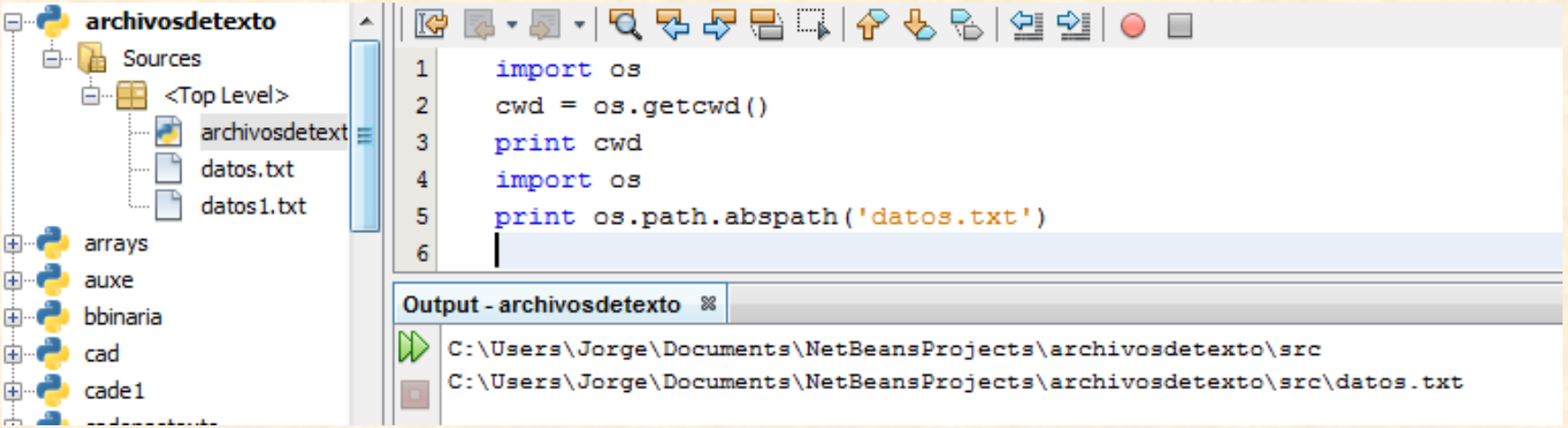
1.- Abrir el archivo de texto
2.- Leer o escribir información sobre el archivo de texto
3.- Cerrar el archivo de texto

1. *Abrir* el fichero indicando su ruta (relativa o absoluta) y el modo de trabajo. Hay varios modos de trabajo:
   - *Lectura*: es posible leer información del fichero, pero no modificarla ni añadir nueva información.
   - *Escritura*: sólo es posible escribir información en el fichero. Por regla general, la apertura de un fichero en modo escritura borra todo el contenido previo del mismo.
   - *Lectura/escritura*: permite leer y escribir información del fichero.
   - *Adición*: permite añadir nueva información al fichero, pero no modificar la ya existente.
2. *Leer o escribir* la información que desees.
3. *Cerrar* el fichero.

Es importante que sigas siempre estos tres pasos. Es particularmente probable que olvides cerrar el archivo, pues Python no detectará esta circunstancia como un fallo del programa.

CAPÍTULO 8 – Persistencia de Datos

Los archivos se organizan en directorios y subdirectorios. Todo programa en ejecución tiene un “directorio actual“. Existen 2 tipos de paths: relativos y absolutos



Ejemplo 1: Lectura de las líneas de un archivo de texto




```
# Paso 1: abrir el archivo
archivo = open('datos.txt','r')
# Paso 2: leer los datos del archivo.
for linea in  archivo:
    print linea
# Paso 3: cerrar el archivo
archivo.close()
```

```
raul

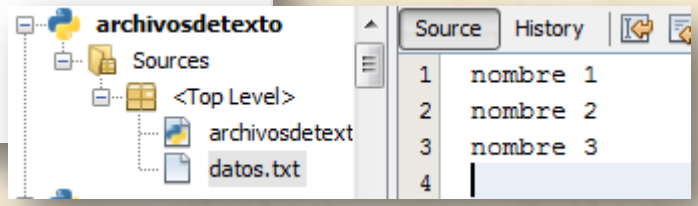
pedro

juan
```

Ejemplo 2: Escritura de las líneas de un archivo de texto




```
# Paso 1: abrir el archivo en modo escritura
archivo = open('datos.txt','w')
# Paso 2: escribir sobre el archivo 3 lineas nueva
archivo.write("nombre 1\n")
archivo.write("nombre 2\n")
archivo.write("nombre 3\n")
# Paso 3: cerrar el archivo
archivo.close()
```



Algunas diferencias:

1.- Cuando abro el archivo en modo escritura “w”, los datos que existían antes en el archivo son destruidos y reemplazados por los nuevos valores ingresados.

2.- Cuando abro el archivo en modo lectura “r”, el archivo debe estar creado físicamente previamente, caso contrario se produce un error.

¿Cómo creo un archivo y cómo agrego datos a un archivo existente?

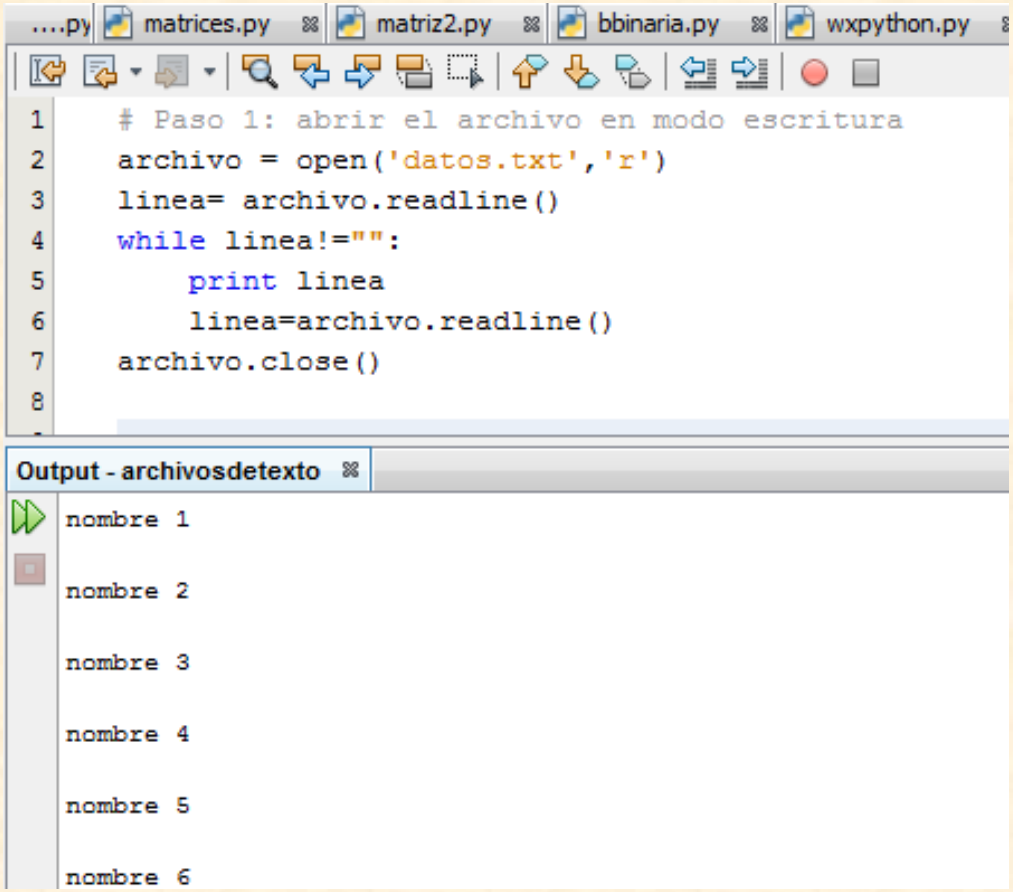
Crear un archivo:

```
# Paso 1: abrir el archivo en modo escritura
archivo = open('datos.txt','w')
# Paso 3: cerrar el archivo
archivo.close()
```

Agregar datos a un archivo:

```
# Paso 1: abrir el archivo en modo escritura
archivo = open('datos.txt','a')
# Paso 2: escribir sobre el archivo 3 lineas nuevas
archivo.write("nombre 4\n")
archivo.write("nombre 5\n")
archivo.write("nombre 6\n")
# Paso 3: cerrar el archivo
archivo.close()
```

Ejemplo 3: Lectura línea a línea de un archivo de texto



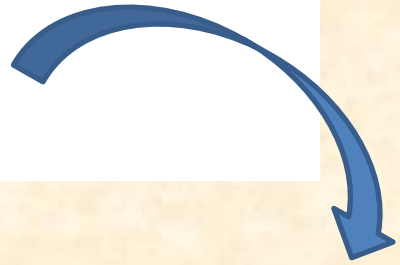
La clase file tiene el método **readline()** que retorna toda una línea del archivo de texto y deja posicionado el puntero de archivo en la siguiente línea.

Cuando llega al final del archivo readline retorna un string vacío.

Ejemplo 4: Lectura de las líneas de un texto

Podemos leer todo el contenido de un archivo de texto y almacenarlo en una **lista** (esto tiene sentido si el archivo de texto no es muy grande). Usamos el método **readlines()** en lugar de readline()

```
# Paso 1: abrir el archivo en modo escritura
archivo = open('datos.txt','r')
lineas= archivo.readlines()
print lineas
archivo.close()
```



```
['raul\n', 'pedro\n', 'juan\n']
```

3.- Control de errores

Si tratas de abrir en modo lectura un archivo inexistente, obtienes un error y la ejecución del programa aborta. Tienes dos posibilidades para reaccionar a esta eventualidad y evitar el fin de ejecución del programa.

1.- Una opción es preguntar antes si el fichero existe.

```
import os
if os.path.exists ('datos3.txt'):
    archivo = open('datos3.txt','r')
    for linea in archivo:
        print linea
    archivo.close()
else:
    print "El archivo no existe"
```

3.- Control de errores

2.- Otra opción es capturar la excepción que genera la apertura:

```
try:
    archivo = open('datos3.txt','r')
    for linea in archivo:
        print linea
    archivo.close()
except IOError:
    print "El archivo no existe"
```

3.- Control de errores

Si queremos eliminar los saltos de línea:

```
archivo = open('datos.txt', 'r')
for linea in  archivo:
    if linea[-1] == '\n':
        linea = linea[:-1]
    print linea
archivo.close()
```

```
raul

pedro

juan
```



```
raul
pedro
juan
```

4.- Ejercicio

Desarrollar un programa que calcule el número de líneas de un archivo de texto.

```
1    nombre = raw_input('Nombre del fichero: ')
2    archivo = open(nombre, 'r')
3    cont= 0
4    for linea in archivo:
5            cont= cont +1
6    archivo.close()
7    print cont
8
```

Output - archivosdetexto

```
Nombre del fichero: datos.txt
3
```

5.- Ejercicios propuestos

1.- Diseña un programa que cuente el número de caracteres de un archivo de texto, incluyendo los saltos de línea. (El nombre del archivo se pide al usuario por teclado.)

2.- Diseñe un programa que, dada una palabra y un nombre de archivo, diga si la palabra aparece o no en el archivo. (El nombre del archivo y la palabra se pedirán al usuario por teclado.)

3.- Diseñe un programa que, dado un nombre de archivo, muestre cada una de sus líneas precedida por su numero de línea. (El nombre del archivo se pedirá al usuario por teclado.)

4.- Diseña un programa que obtenga los 100 primeros números de la serie de Fibonnaci y los almacene en un archivo de texto llamado fibo.txt.

5.- Ejercicios propuestos

5.- Diseñar un programa que me permita hacer lo siguiente:

Se necesita llevar el registro de la cantidad de los productos que ingresan a la tienda "Paquito", el precio de los productos y los totales de los productos facturados. Para esto el programa almacenará los datos en 3 archivos de texto:

1.- Productos (almacena código, producto, pvp)
2.- Stock (almacena código, cantidad de producto)
3.- Factura (almacena número de factura y el total vendido)

Archivo: Productos

codigo	producto	pvp
1	arroz	12
2	papas	3
3	galletas	44
4	leche	22

Archivo: Stock

codigo	cantidad
1	22
2	33
3	44
4	5

Archivo: Factura

numero	total
1	323
2	454
3	123
4	22

5.- Ejercicios propuestos

El programa debe permitir agregar un nuevo producto y la cantidad del mismo. También se podrá agregar unidades a un producto ya existente. El programa debe permitir actualizar el precio de un producto.

El programa debe tener una opción que diga facturar productos en la que se deberán escoger los productos que deberán facturarse (controlar que el producto esté en stock). Cuando se lleve a cabo esto se debe calcular el total de la factura con los productos, calcular el total y generar un número nuevo de factura que deberá guardarse en un archivo de texto.

El programa debe permitir entre sus opciones:

a) Generar un informe del precio de un producto cualquier por su código
b) Informe con el número total de factura y la suma de sus totales
c) Informe con los producto que tengan un stock menor a 10 unidades